

Tribhuvan University
IOST
Prithvi Narayan Campus



त्रिभुवन विश्वविद्यालय
पृथ्वीनारायण क्याम्पस

Lab Report of Software Engineering by:

Vivek Paudel

Sixth Semester

Section: A

Roll no: 80012185 (Class: 24)

Submitted To:

Neelam Thapa

(SE Teacher)

INDEX

Name: Vivek Paudel

Roll No: 80012185 (24)

SN	Name of Experiment	Date of submission	Signature
1	Study of Software Types and Attributes of Good Software		
2	Case Study on Software Process Models		
3	Agile Development Simulation		
4	Requirements Analysis		
5	UML System Modeling		
6	Architectural Design		
7	Object-Oriented Design using UML		
8	Testing Techniques		
9	Evolution and Maintenance		
10	Project Management Exercise		

Experiment 1: Study of Software Types and Attributes of Good Software

1. Objective

The primary objective of this experiment is to identify and understand different types of software and to analyze the key attributes that constitute good software. This involves evaluating real-world software systems based on characteristics such as usability, reliability, maintainability, and efficiency.

2. Theory

2.1 Software Types

Software can be broadly categorized based on its function and application. The main types are:

- **System Software:** Acts as the platform for other software, manages computer hardware, and provides basic functionalities. Examples include Operating Systems (Linux, macOS, Android), device drivers, and utility software.
- **Application Software:** End-user software designed to perform specific tasks. Examples include Spotify, Adobe Illustrator, Zoom, and Slack.
- **Programming Software:** A set of tools that assist developers in writing, debugging, and building code. Examples include compilers (GCC), debuggers (GDB), and IDEs like PyCharm or IntelliJ IDEA.
- **Embedded Software:** Written to control devices or hardware systems that are not traditional computers. Examples include software in smart washing machines, drone flight controllers, and pacemaker devices.

2.2 Attributes of Good Software

The quality of a software product is determined by several key attributes:

- **Usability:** The ease with which users can learn and navigate the application.
- **Reliability:** The consistency of software performance under varying conditions without crashing or data corruption.
- **Maintainability:** The effort required to update code, fix defects, or adapt to new operational environments.
- **Efficiency:** The optimal usage of hardware resources like system RAM, GPU/CPU cycles, and network bandwidth.

3. Procedure

1. Select two distinct real-world software applications (Spotify and Zoom) for analysis.
2. Identify the software type for each system based on its primary function.
3. Evaluate each software against usability, reliability, maintainability, and efficiency.
4. Record and organize findings into a comparative table.
5. Draw conclusions based on the comparative study.

4. Practical Work

Analysis of Spotify and Zoom

Attribute	Spotify	Zoom
Software Type	Application Software (Digital Music Service)	Application Software (Video Conferencing Platform)
Usability	High; clean layout with cross-device synchronization and effortless playback controls.	High; one-click meeting joins and simple screen-sharing setup.
Reliability	High; persistent offline caching prevents playback interruptions during connectivity drops.	High; adaptive video encoding preserves audio calls during network degradation.
Maintainability	High; decoupled microservices allow background feature deployment without downtime.	High; modular architecture supports rapid deployment of security patches.
Efficiency	High; lightweight audio streaming buffers efficiently on minimal memory footprints.	Moderate to High; video compression minimizes bandwidth usage but uses substantial CPU/GPU resources.

5. Conclusion

Evaluating Spotify and Zoom demonstrates that domain context dictates attribute prioritization. Streaming media applications focus on memory efficiency and persistent playback, while real-time communication software prioritizes network adaptability and low packet latency.

Experiment 2: Case Study on Software Process Models

1. Objective

To understand, apply, and compare different software process models by modeling the development of an **IoT Smart Home Automation System** using the Waterfall, Incremental, and Prototyping models.

2. Theory

2.1 Software Process Models

A software process model outlines the phases, tasks, and flow involved in creating software.

- **Waterfall Model:** A strict sequential development approach where each phase completes fully before the next begins.
- **Incremental Model:** Releases functional core features first, adding progressive functional layers across subsequent cycles.
- **Prototyping Model:** Rapidly constructs functional prototypes early to clarify ambiguous client expectations before committing to final production code.

3. Procedure

1. Define the functional baseline of an IoT Smart Home Automation System.
2. Conceptualize development stages under Waterfall, Incremental, and Prototyping process models.
3. Map architectural workflows and construct diagram representations for each lifecycle.
4. Compare model traits across key operational variables.

4. Practical Work

4.1 Problem Description

An IoT Smart Home Automation System enables users to remotely control household lighting, HVAC systems, security locks, and power outlets via a mobile application, utilizing automated schedule rules and sensor inputs.

4.2 Process Model Applications

- **Waterfall Model Application:**
All hardware integrations, communication protocols, sensor configurations, and user interface designs must be fully locked during the initial requirements analysis.
Development moves strictly from architectural design to implementation, testing, and deployment.

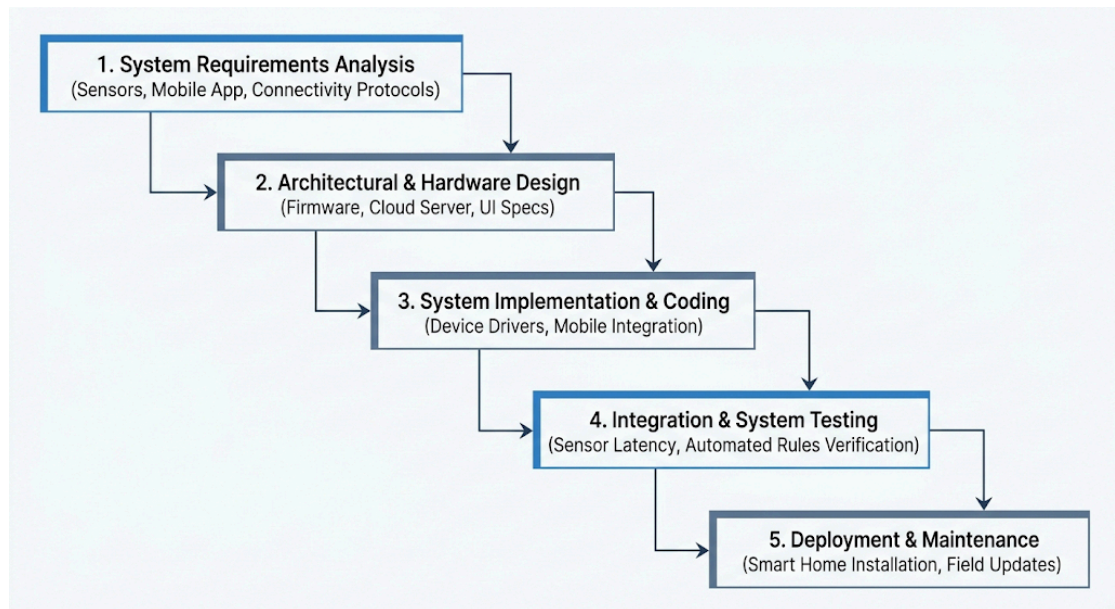


Fig. Waterfall Model Flowchart for Smart Home Automation System

- **Incremental Model Application:**

Development proceeds via functional iterations:

1. *Increment 1:* Basic remote light switch toggling and device connectivity logic.
2. *Increment 2:* Smart thermostat management and automated sensor integration (motion/temperature).
3. *Increment 3:* Security lock integration, user automation routines, and push notifications.

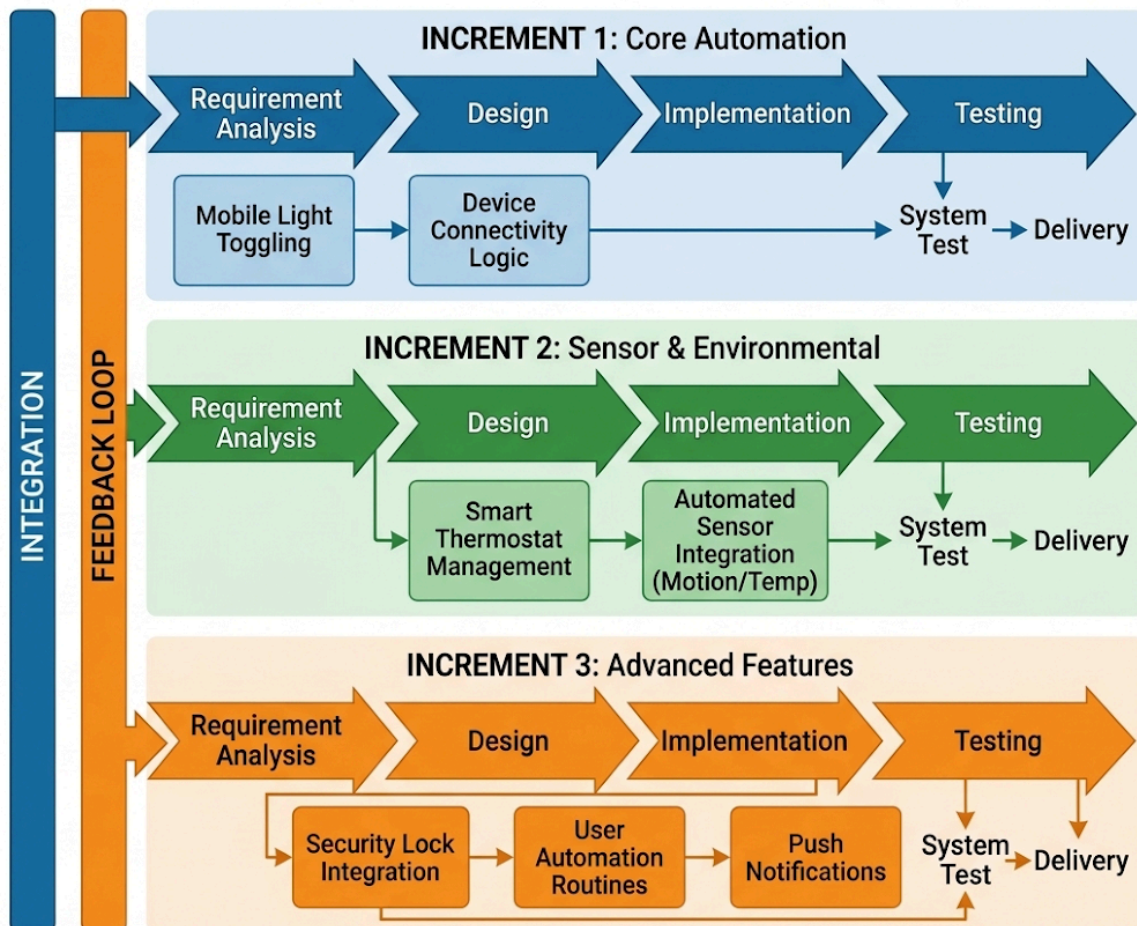


Fig. Incremental Model Architecture Diagram for IOT smart home

- **Prototyping Model Application:**

1. Gather early user feedback regarding mobile dashboard interactions.
2. Build a low-code interactive GUI prototype connected to mock sensor hardware.
3. Perform usability trials with users to refine layout preferences and automated control logic.
4. Finalize production code for mobile and firmware integration.

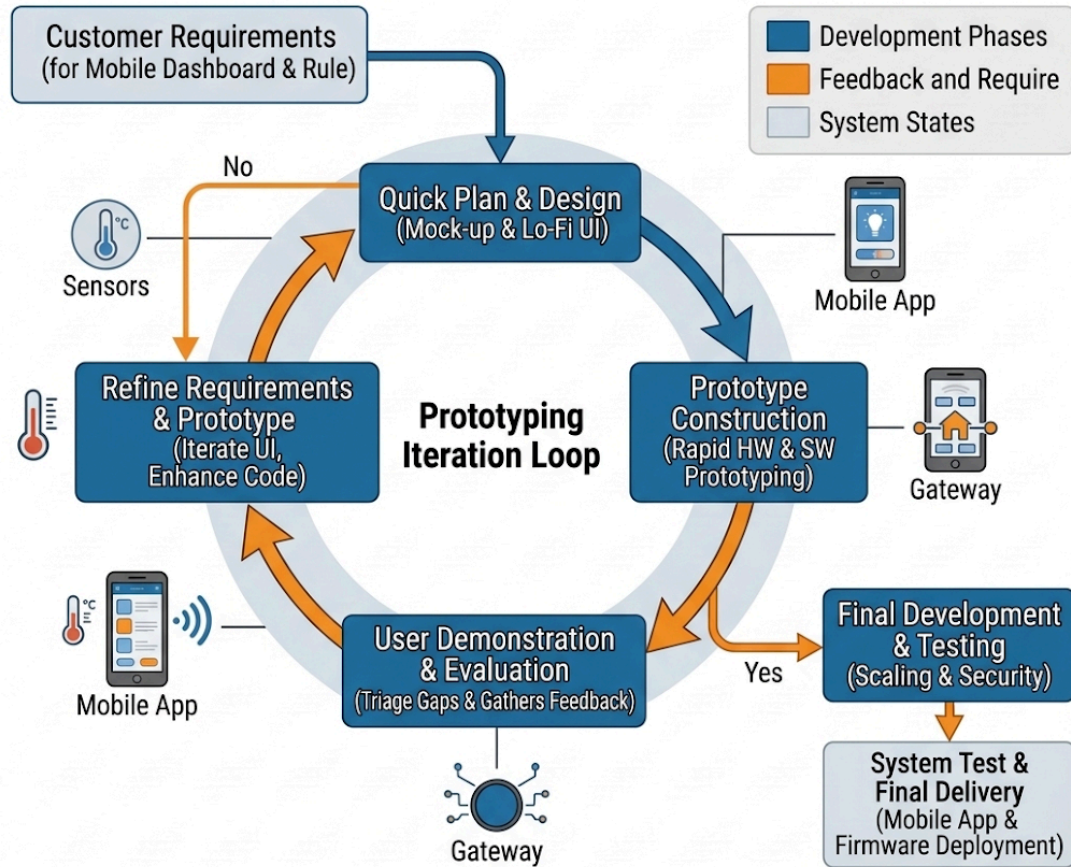


Fig. Prototyping Model Iteration Diagram for IoT Smart Home System

4.3 Comparison Table

Feature / Model	Waterfall	Incremental	Prototyping
Flexibility	Low	High	Very High
User Involvement	Low (mainly initial stage)	Moderate to High	Continuous across iterations
Risk Management	Low	Moderate to High	High (detects UI/hardware gaps early)

Time to First Output	Long	Short (per increment release)	Very Short
Hardware/Firmware Fit	Moderate	High	Ideal

5. Conclusion

For an IoT Smart Home platform, strict Waterfall models carry high integration risks. Combining Prototyping to validate dashboard designs with an Incremental strategy for multi-device firmware deployment balances release speed with system stability.

Experiment 3: Agile Development Simulation

1. Objective

To understand Agile principles and Scrum by simulating key activities for a **Ride-Sharing Platform**, including a Product Backlog, Sprint Backlogs, and a Scrum Board.

2. Theory

2.1 Agile and Scrum

- **Agile:** An iterative design mindset delivering functional software rapidly in small updates.
- **Scrum:** An Agile framework that breaks execution down into fixed-length cycles known as Sprints.

2.2 Core Artifacts

- **Product Backlog:** A prioritized catalog of features written as user stories.
- **Sprint Backlog:** The specific work items selected from the backlog for completion during the active Sprint.
- **Scrum Board:** A visual task board divided into *To Do*, *In Progress*, and *Done*.

3. Procedure

1. Define user scenarios for a Ride-Sharing Platform.
2. Build a prioritized Product Backlog of user stories.
3. Plan Sprints and select target work items.
4. Render progress on a visual Scrum Board.

4. Practical Work

4.1 Product Backlog: Ride-Sharing Platform

ID	User Story	Priority
PBI-01	As a passenger, I want to create an account and register a payment method.	Must Have
PBI-02	As a passenger, I want to set pickup/drop-off points on a map.	Must Have
PBI-03	As a driver, I want to accept or decline incoming ride requests.	Must Have
PBI-04	As a passenger, I want to track my assigned driver's location in real time.	Must Have

PBI-05	As a passenger, I want to receive ride cost estimates before booking.	Should Have
PBI-06	As a user, I want to rate drivers/passengers after trip completion.	Should Have
PBI-07	As a passenger, I want to schedule rides for future dates.	Could Have
PBI-08	As a passenger, I want to split fares with other passengers.	Could Have

4.2 Scrum Board State

Ride sharing App

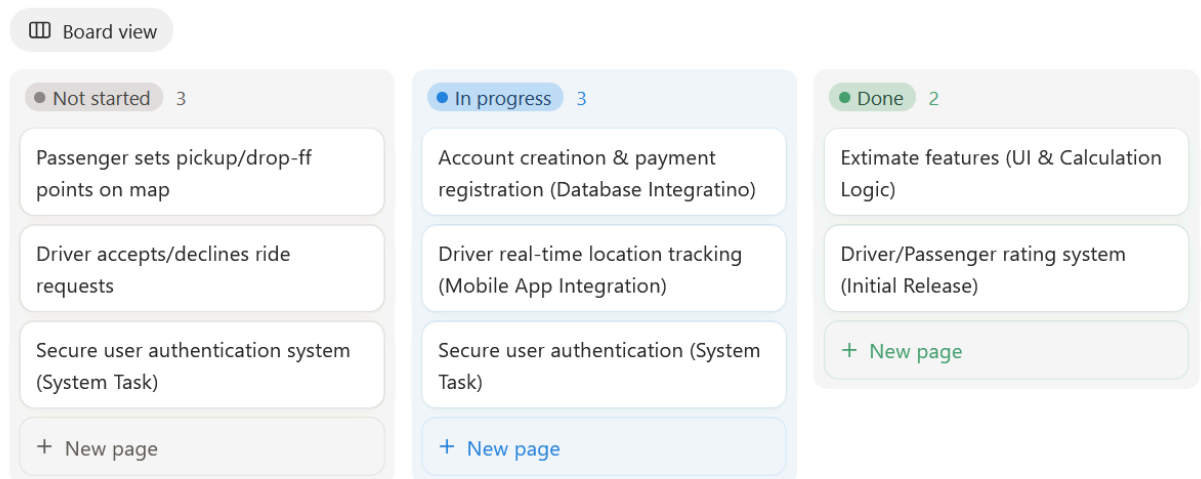


Fig. Scrum Board of Ride Sharing App Image with To Do, In Progress, and Done Columns

5. Conclusion

Simulating Scrum practices demonstrates how breaking down complex applications into bite-sized user stories enables steady feature delivery and rapid response to market shifts.

Experiment 4: Requirements Analysis

1. Objective

Practice requirements identification by creating a Software Requirements Specification (SRS) for an **Hospital Emergency Room Patient Management System**, defining functional and non-functional requirements, and constructing a Use Case diagram.

2. Theory

- **SRS Document:** A specification detailing system behavior, boundaries, and operational constraints.
- **Functional Requirements:** Define explicit capabilities and user actions.
- **Non-Functional Requirements:** Define operational performance, security standards, and availability criteria.
- **Use Case Diagram:** Structural UML illustration displaying system features mapped to interacting roles.

3. Practical Work: SRS Table

Requirement Type	ID	Requirement Details
Functional	FR-1	Multi-Factor Authentication for Triage Staff, Doctors, and System Administrators.
Functional	FR-2	Triage check-in routine to log patient vital signs and assign priority codes.
Functional	FR-3	Real-time bed tracking and room allocation tracking modules.
Functional	FR-4	Digital prescription issuance and pharmacy order dispatch integration.
Non-Functional	NFR-1	Latency under 1 second for emergency alert broadcasts; 99.99% operational uptime.

Non-Functional	NFR-2	Full HIPAA compliance with end-to-end encrypted medical record storage.
Non-Functional	NFR-3	High-contrast UI optimized for touchscreen clinical tablets.
Non-Functional	NFR-4	Automated transaction synchronization across local and cloud backup node locations.

4.1 System Use Case Diagram

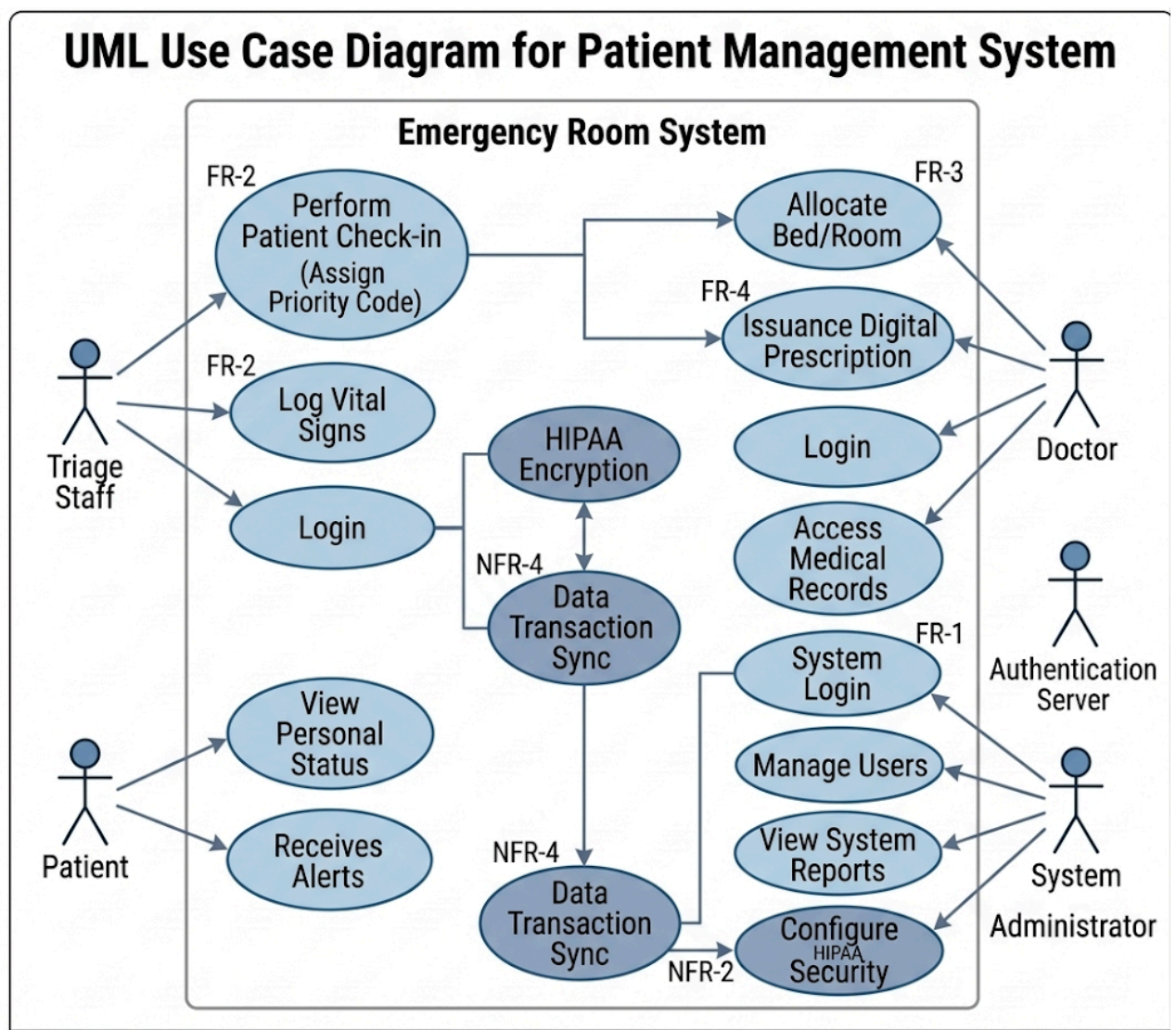


Fig. UML Use Case Diagram for Emergency Room System

5. Conclusion

Drafting precise functional and non-functional requirements for critical software reduces runtime failures and prevents requirement creep during downstream development.

Experiment 5: UML System Modeling

1. Objective

Practice system modeling using UML for an **Automated Library Management System** through Context, Use Case, Sequence, and Activity diagrams.

2. Theory

- **Context Diagram:** High-level boundary diagram showing system exchanges with external actors.
- **Use Case Diagram:** Structural map displaying actor access rights to system functionalities.
- **Sequence Diagram:** Detailed timeline detailing sequential function calls and responses across system objects.
- **Activity Diagram:** Logical flowchart mapping conditional branches, processes, and state changes.

3. Practical Work

3.1 Diagrams and Workflows

- **Context Diagram:** Captures external communications between Members, Librarians, and the core system.

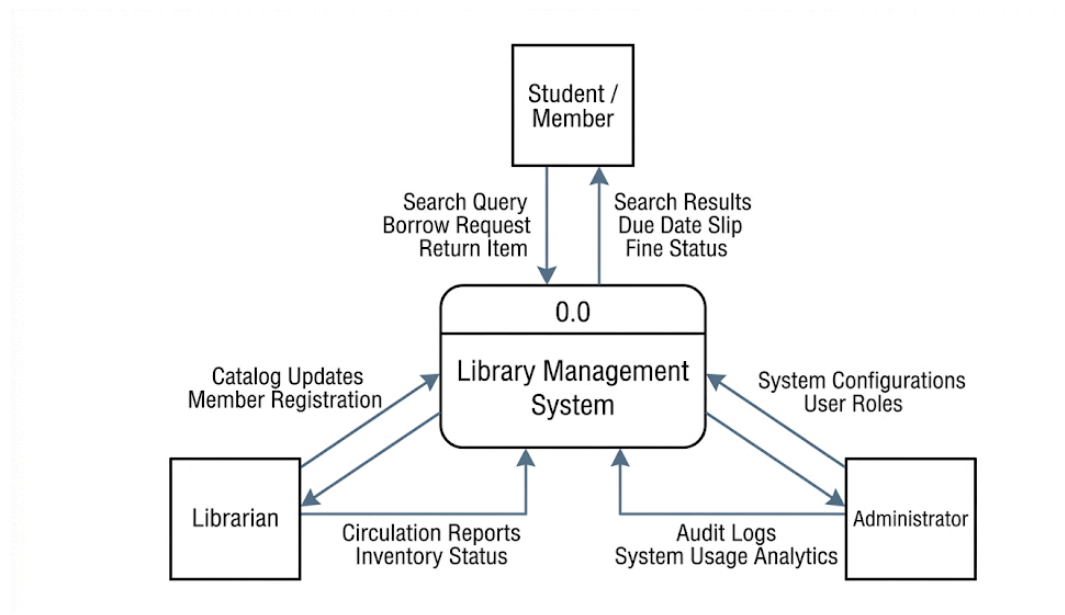


Fig. Context Diagram for Library Management System

- **Use Case Diagram:** Models borrowing routines, inventory indexing, fine management, and user creation.

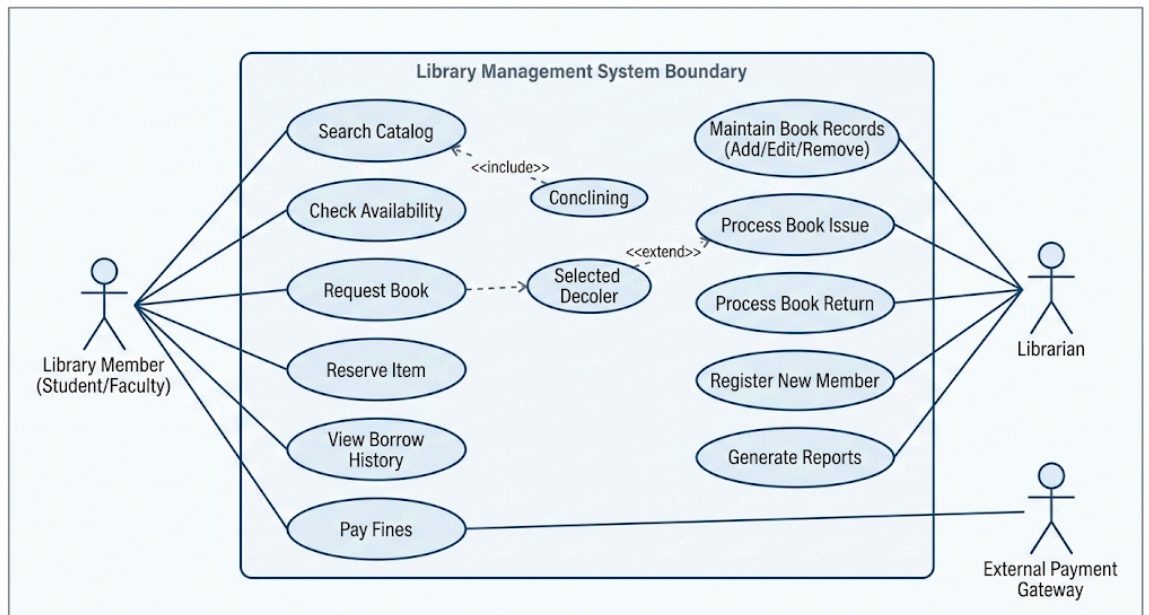


Fig. Use Case Diagram for Library Management System

- **Sequence Diagram ("Reserve Book"):** Maps request validation, inventory availability checks, status updates, and confirmation messaging.

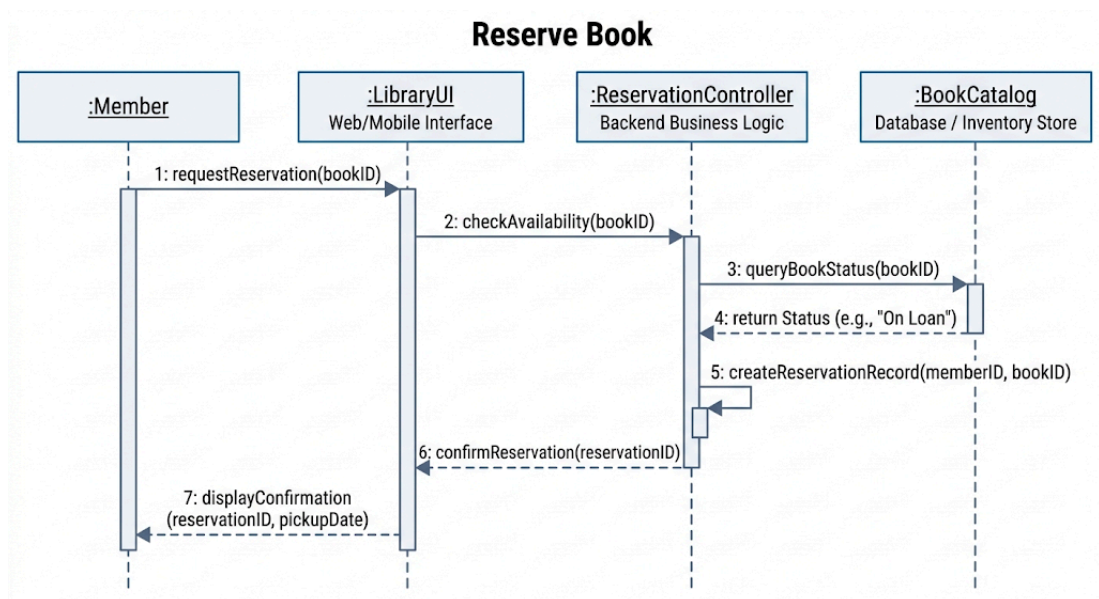


Fig. Reserve Book Scenario

- **Activity Diagram ("Return Borrowed Item"):** Maps the item intake process, overdue calculations, fee processing, and shelf status updates.

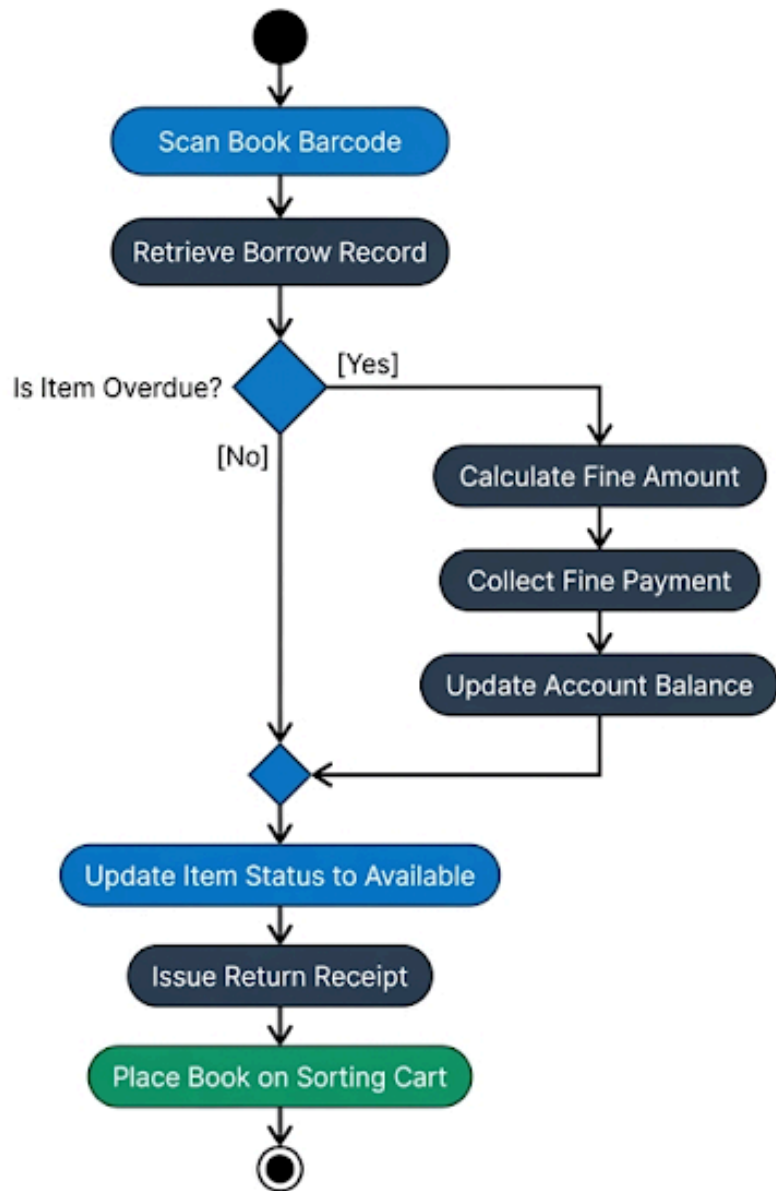


Fig. Activity Diagram - Return Borrowed Item Workflow

4. Conclusion

UML models provide a comprehensive blueprint for software systems, clarifying static structures and dynamic workflows for engineering teams.

Experiment 6: Architectural Design

1. Objective

Explore and understand software architectural patterns applied to a **SaaS Learning Management System (LMS)**.

2. Theory

- **Layered Architecture:** Segregates system responsibilities into Presentation, Logic, Data Access, and Database tiers.
- **Client-Server Architecture:** Connects distributed user client interfaces with centralized server resources.
- **MVC Pattern:** Isolates core data models, UI templates, and routing controllers.

3. Practical Work

3.1 Architectural Layouts

- **Layered Architecture Diagram:**

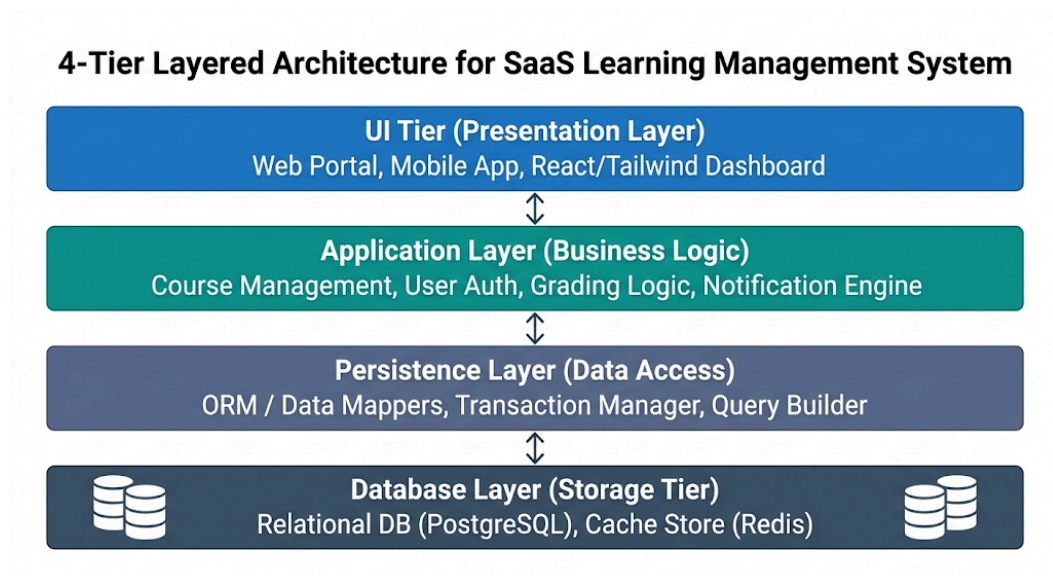


Fig. Layered Architecture Diagram

- **Client-Server Flow Diagram:**

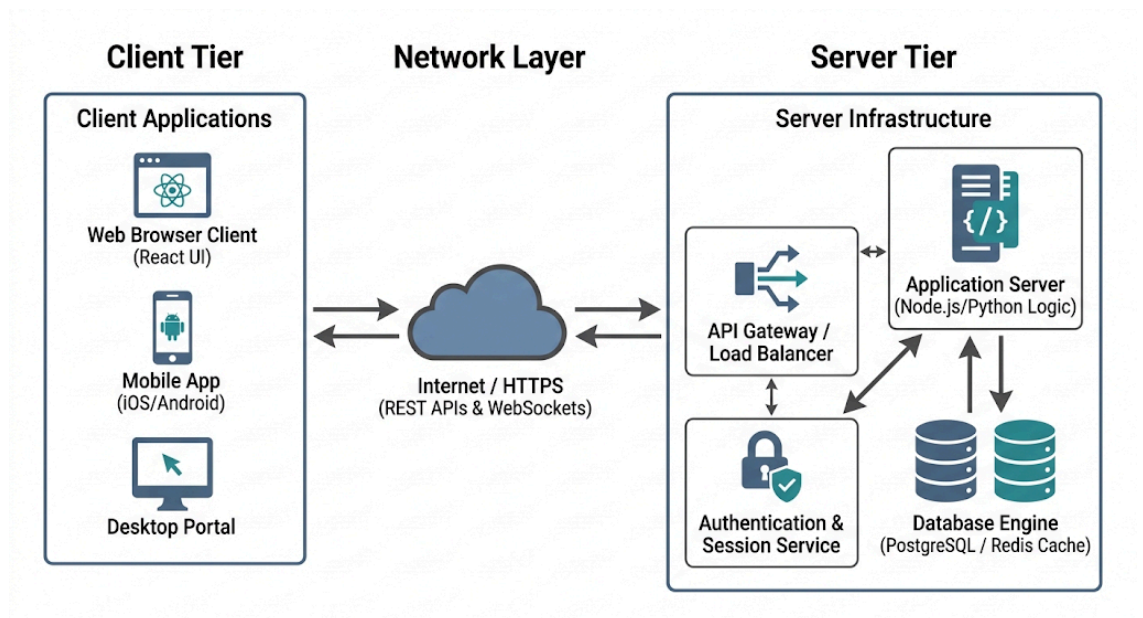


Fig. Client-Server Architecture Diagram

- **MVC Pattern Diagram:**

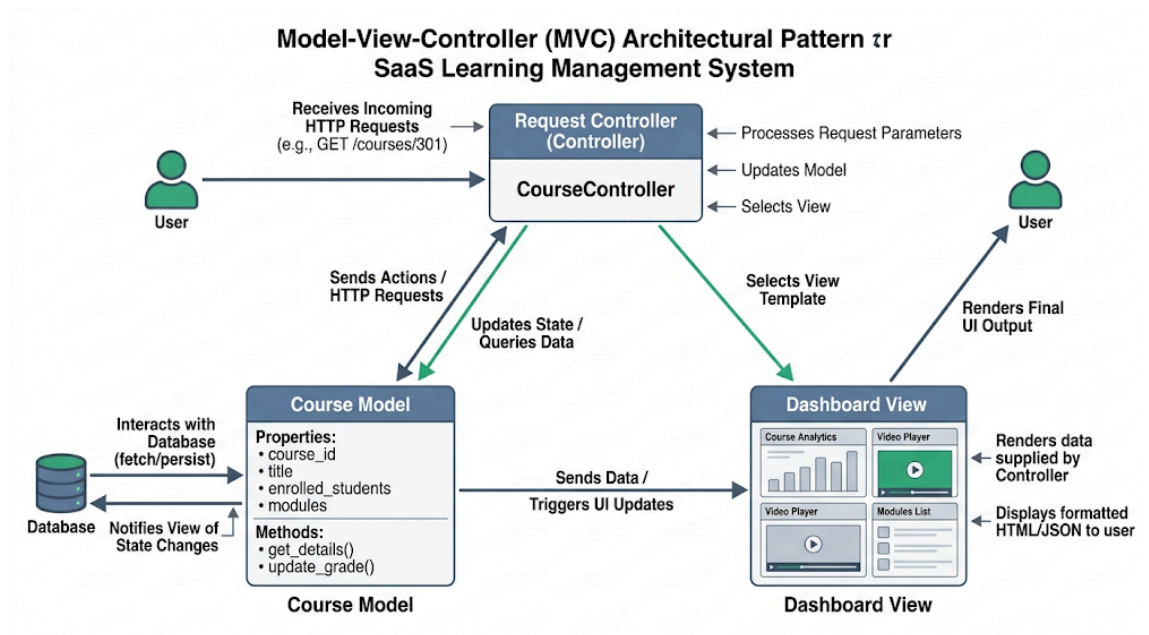


Fig. MVC Architecture Diagram

4. Conclusion

Decoupling application logic into structured architectural patterns ensures scalability, maintainability, and clean code organization.

Experiment 7: Object-Oriented Design using UML

1. Objective

Apply Object-Oriented Design (OOD) principles to model and implement a **Hotel Reservation System**.

2. Theory

- **Class Diagram:** Visual model depicting object classes, attributes, methods, and structural relationships.
- **Object Diagram:** Runtime snapshot of instantiated objects and active attribute values.

3. Practical Work

3.1 UML Modeling

- **Class Diagram:**

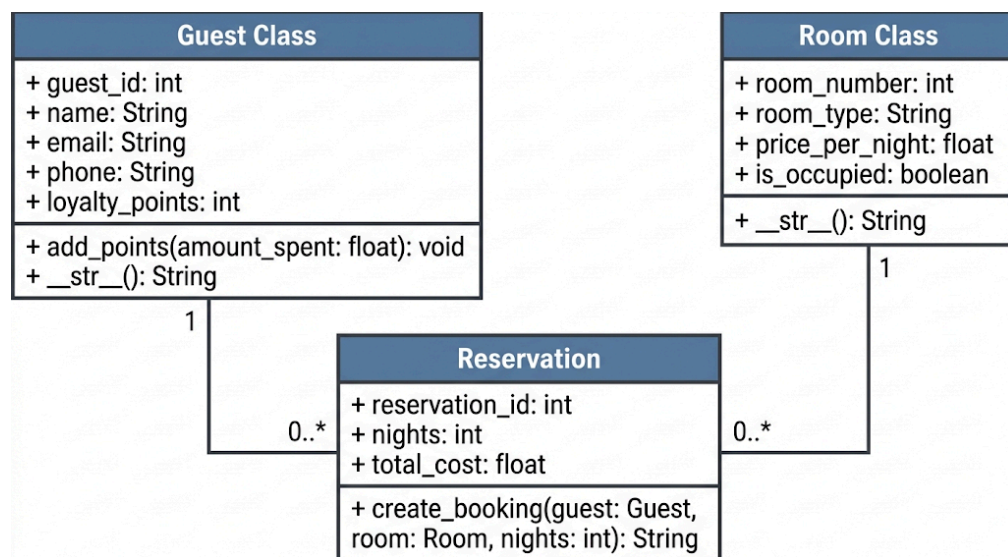


Fig. Class Diagram for Hotel Reservation System

- **Object Diagram:**

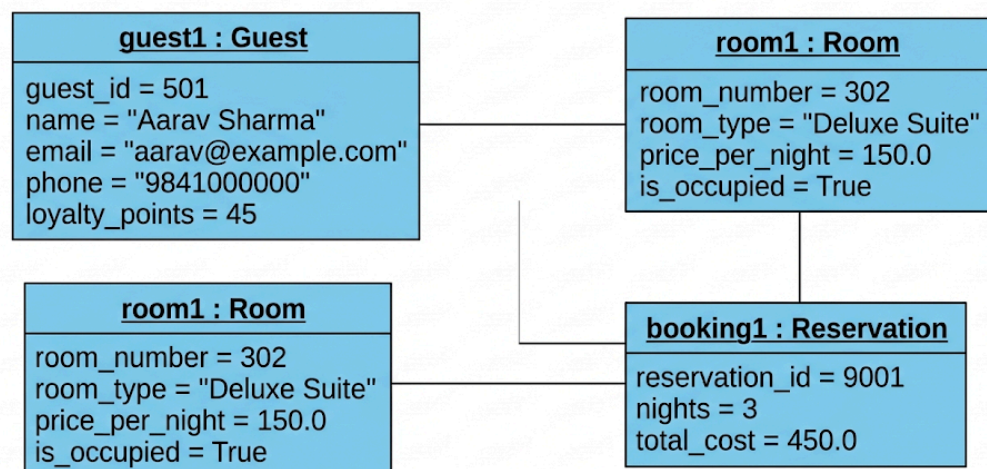


Fig. Object Diagram showing runtime booking instances

3.2 Python Implementation

Python

class Guest:

```
def __init__(self, guest_id, name, email, phone):
```

```
    self.guest_id = guest_id
```

```
    self.name = name
```

```
    self.email = email
```

```
    self.phone = phone
```

```
def __str__(self):
```

```
    return f"Guest[ID={self.guest_id}, Name={self.name}, Email={self.email}]"
```

class Room:

```
def __init__(self, room_number, room_type, price_per_night):
```

```
    self.room_number = room_number
```

```
    self.room_type = room_type
```

```
    self.price_per_night = price_per_night
```

```
    self.is_occupied = False
```

```
def __str__(self):
```

```
    return f"Room[No={self.room_number}, Type={self.room_type},
```

```
Rate=${self.price_per_night}]"
```

class Reservation:

```
def create_booking(self, guest, room, nights):
```

```
    if not room.is_occupied:
```

```
        room.is_occupied = True
```

```
        total_cost = room.price_per_night * nights
```

```
        print(f"Booking confirmed for {guest.name} in Room {room.room_number}.")
```

```
        return f"Reservation Success: Total Cost = ${total_cost}"
```

```
    else:
```

```
        return f"Reservation Failed: Room {room.room_number} is occupied."
```

```
if __name__ == "__main__":
```

```
    guest1 = Guest(501, "Aarav Sharma", "aarav@example.com", "9841000000")
```

```
    room1 = Room(302, "Deluxe Suite", 150.0)
```

```
    booking_service = Reservation()
```

```
    result = booking_service.create_booking(guest1, room1, nights=3)
```

```
    print(guest1)
```

```
    print(room1)
```

```
    print(result)
```

4. Conclusion

Designing systems with OOD principles leads to modular codebases where real-world domain entities map directly to software classes.

Experiment 8: Testing Techniques

1. Objective

Understand and execute fundamental software testing techniques, including Unit, Black Box, and White Box testing on an **E-Commerce Discount & Tax Processor**.

2. Theory

- **Unit Testing:** Tests individual functions in isolation to verify internal correctness.
- **Black Box Testing:** Verifies functional outputs against specified requirements without inspecting code logic.
- **White Box Testing:** Analyzes internal execution paths, conditional loops, and logic branches.

3. Practical Work

3.1 System Under Test & Unit Tests

Python

```
# pricing.py
```

```
def calculate_discount(price, is_member):  
    if is_member:  
        return price * 0.80 # 20% discount for members  
    return price
```

```
def apply_tax(price, tax_rate=0.13):  
    return price * (1 + tax_rate) # 13% tax default
```

Python

```
# test_pricing.py
```

```
import unittest  
from pricing import calculate_discount, apply_tax
```

```
class TestPricingSystem(unittest.TestCase):  
    def test_calculate_discount(self):  
        self.assertEqual(calculate_discount(100.0, True), 80.0)  
        self.assertEqual(calculate_discount(100.0, False), 100.0)  
  
    def test_apply_tax(self):  
        self.assertAlmostEqual(apply_tax(100.0, 0.13), 113.0)  
        self.assertAlmostEqual(apply_tax(200.0, 0.0), 200.0)
```

```
if __name__ == "__main__":  
    unittest.main()
```

3.2 Black Box Test Suite

Input Price	Membership Flag	Expected Output	Actual Behavior
\$100.00	True	\$80.00	Pass
\$100.00	False	\$100.00	Pass
\$50.00	True	\$40.00	Pass

3.3 White Box Logical Path Suite (calculate_discount Routine)

Target Path	Input (price, is_member)	Logic Condition Hit	Expected Output
Path A	100.0, True	is_member == True branch executed	80.0
Path B	100.0, False	is_member == False branch executed	100.0

4. Conclusion

Combining unit testing with black-box validation and white-box path analysis ensures both code logic and business requirements are thoroughly verified.

Experiment 9: Evolution and Maintenance

1. Objective

Practice software evolution by upgrading the **Hotel Reservation System** from Experiment 7 to include a **Loyalty Points Reward Module**.

2. Theory

- **Corrective Maintenance:** Fixing bugs or defects in production code.
- **Adaptive Maintenance:** Modifying code to support new environments or platforms.
- **Perfective Maintenance:** Adding new features or enhancing existing performance.
- **Preventive Maintenance:** Refactoring structural design to improve future maintainability.

3. Practical Work

3.1 Evolved Python Module

Python

class Guest:

```
def __init__(self, guest_id, name, email, phone):
    self.guest_id = guest_id
    self.name = name
    self.email = email
    self.phone = phone
    self.loyalty_points = 0 # Added Feature: Loyalty Tracking

def add_points(self, amount_spent):
    points_earned = int(amount_spent // 10) # 1 point per $10 spent
    self.loyalty_points += points_earned
    print(f"Added {points_earned} loyalty points to {self.name}'s account.")

def __str__(self):
    return f"Guest[ID={self.guest_id}, Name={self.name}, Points={self.loyalty_points}]"
```

class Room:

```
def __init__(self, room_number, room_type, price_per_night):
    self.room_number = room_number
    self.room_type = room_type
    self.price_per_night = price_per_night
    self.is_occupied = False

def __str__(self):
    return f"Room[No={self.room_number}, Type={self.room_type}, Rate=${self.price_per_night}]"
```

class Reservation:

```
def create_booking(self, guest, room, nights):
    if not room.is_occupied:
        room.is_occupied = True
```

```

        total_cost = room.price_per_night * nights
        guest.add_points(total_cost) # Evolved integration
        print(f"Booking confirmed for {guest.name} in Room {room.room_number}.")
        return f"Reservation Success: Total Cost = ${total_cost}"
    else:
        return f"Reservation Failed: Room {room.room_number} is occupied."

if __name__ == "__main__":
    guest1 = Guest(501, "Vivek Paudel", "vivek@gmail.com", "9841000000")
    room1 = Room(302, "Deluxe Suite", 150.0)

    booking_service = Reservation()
    result = booking_service.create_booking(guest1, room1, nights=3)

    print(guest1)
    print(room1)
    print(result)

```

Output:

```

PS D:\E drive\No Antivirus for compilation\SE> py .\one.py
Added 45 loyalty points to Vivek Paudel's account.
Booking confirmed for Vivek Paudel in Room 302.
Guest[ID=501, Name=Vivek Paudel, Points=45]
Room[No=302, Type=Deluxe Suite, Rate=$150.0]
Reservation Success: Total Cost = $450.0
PS D:\E drive\No Antivirus for compilation\SE>

```

3.2 Maintenance Activity Classification

Activity	Description	Maintenance Type
Added loyalty_points attribute	Extended Guest class data structure.	Perfective
Added add_points() method	Implemented point reward logic based on spending.	Perfective

Updated create_booking() workflow	Integrated point calculation into reservation execution.	Perfective
Regression Testing	Validated that room assignment logic remained functional.	Corrective & Perfective

4. Conclusion

Software evolution requires updating existing codebases without breaking legacy capabilities, making regression testing and documentation vital during system maintenance.

Experiment 10: Project Management Exercise

1. Objective

Apply project management techniques by building a Work Breakdown Structure (WBS), calculating cost and effort using the Basic COCOMO model, and creating a Gantt chart for a **FinTech Mobile Wallet Application**.

2. Theory

- **Work Breakdown Structure (WBS):** A hierarchical breakdown of project work into manageable tasks.
- **Basic COCOMO Model:** Estimates software project effort, timeline, and staffing based on size (KLOC):
 - Effort: $E = a * (KLOC)^b$ (Person-Months)
 - Duration: $D = c * (E)^d$ (Months)
 - Staffing Requirements: $P = E/D$ (Persons)

COCOMO Coefficients Table

Project Type	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

3. Practical Work

3.1 Work Breakdown Structure

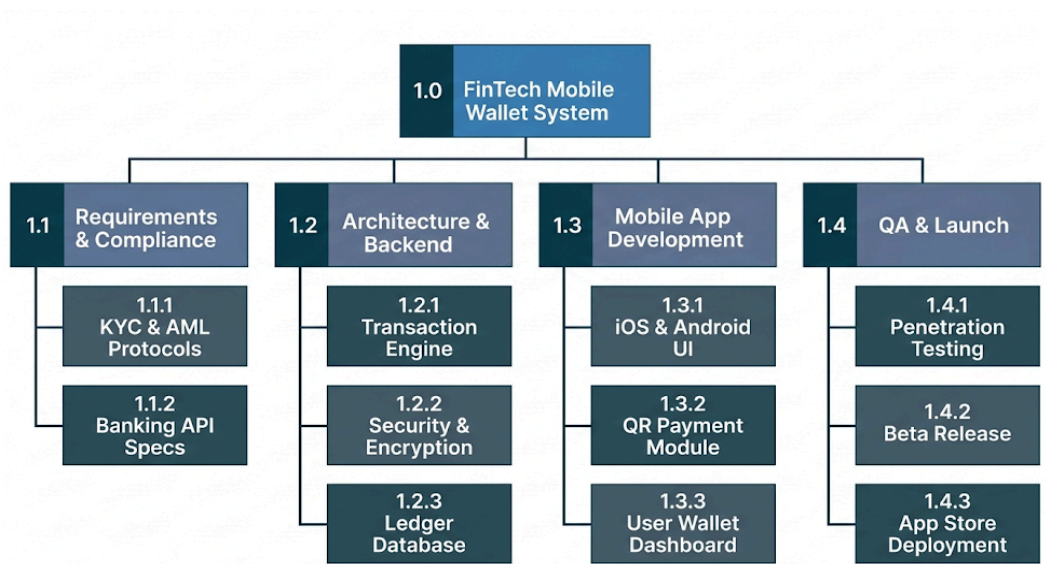


Fig. WBS Hierarchy Chart for FinTech Mobile Wallet

3.2 COCOMO Calculation Summary

- **Input Parameters:**
 - Project Size = 35 KLOC (35,000 Lines of Code)
 - Project Type = Organic (\$a = 2.4, b = 1.05, c = 2.5, d = 0.38\$)
- **Calculations:**

$E = 2.4 * (35^{1.05}) \approx 100.86$ person-months

$D = 2.5 * (100.86^{0.38}) \approx 14.41$ months

$P = 100.86 / 14.41 \approx 7.0$ persons
- **Summary Metrics:**
 - **Effort:** ~101 Person-Months
 - **Schedule:** ~14.4 Months
 - **Average Team Size:** ~7 Developers

3.3 Project Gantt Chart

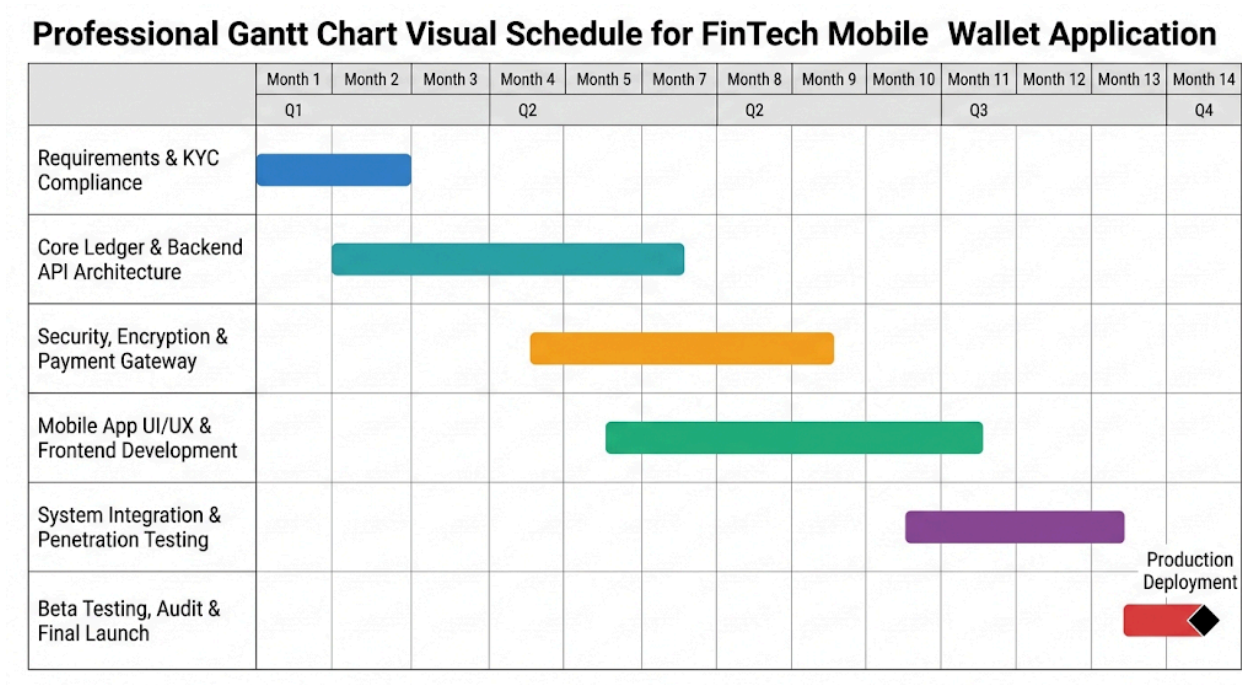


Fig. Project Schedule Gantt Chart covering 14-Month Timeline

4. Conclusion

Combining structured task breakdown (WBS) with COCOMO estimation enables development teams to create realistic schedules and allocate budget resources effectively.